OliveTrace DevOps

Applications Web Avancees - dossier de validation finale

Equipe : Sofiene Zayati, Mariem, Oussema, Hana et Aymen.
Date du dossier : 22 octobre 2026.

1. Objectif et perimetre

OliveTrace suit le parcours de l'huile d'olive tunisienne : origine, recolte et transformation, laboratoire et certification, distribution, puis retours consommateurs. Les cinq modules partagent authentification, roles et gestion des utilisateurs.

Ce dossier presente le processus DevOps, le pipeline du developpement a la CI puis a la livraison/deploiement, les controles, les preuves et les limites reelles de la demonstration.

Element	Mise en place
Application	Laravel 12, PHP 8.4, Blade, Eloquent et MySQL. Breeze pour l'authentification.
Collaboration	GitHub : branches de fonctionnalites, Pull Requests, develop et main.
CI / qualite	GitHub Actions, tests sur SQLite et MySQL, rapports JUnit/Clover, Pint et SonarQube Cloud.
Packaging / CD	Docker multi-stage, PHP/Apache, configuration Railway et migrations avant deploiement.
Exploitation	Health check /up, logs, metriques de la plateforme, stockage persistant et procedure de retour arriere.

Etat verifie au moment de la remise

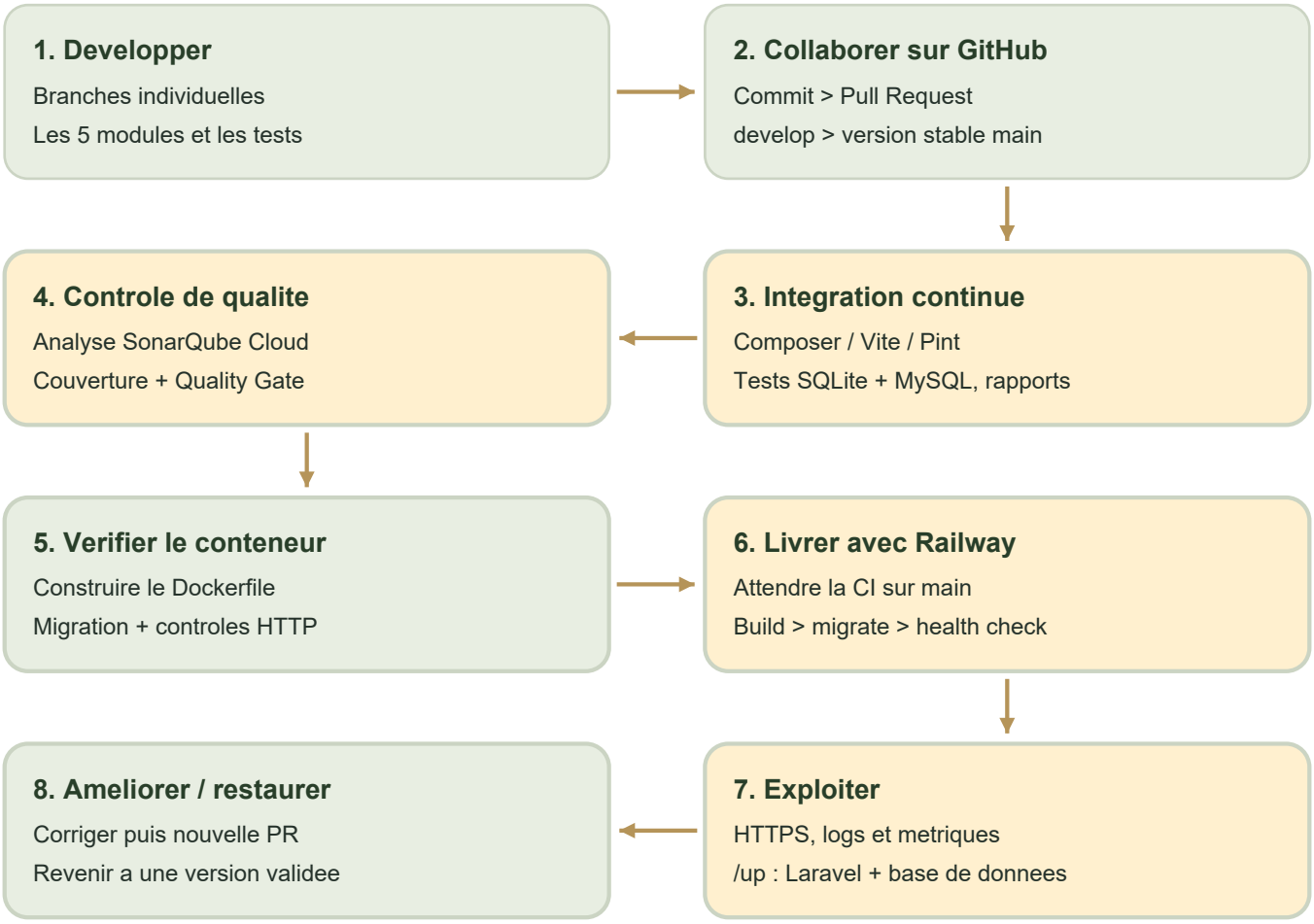
All five modules are integrated. Docker and the local test suites are verified. The upgraded GitHub pipeline is running; Railway deployment is the next step.

Tests locaux : **240 tests et 1 740 assertions reussis**, sur SQLite et sur une base MySQL de test distincte. Image Docker construite et controlee par HTTP. Les tests IA automatises utilisent des reponses simulees.

Version source des preuves : **282811a**. Les modifications de documentation ne changent pas les fonctions metier.

2. Diagramme du processus DevOps

Chaque version passe par des controles reproductibles avant sa promotion. Un echec de test, de Quality Gate ou de verification du conteneur bloque la chaine CI.



Legende : vert = prepare et verifie ; orange = etape dont la verification finale depend encore de la CI ou de l'hebergeur. Les liens de preuve et le statut exact figurent aux pages suivantes.

Developpement et collaboration

Chaque membre modifie sa branche, effectue ses controles locaux et ouvre une PR. L'integration cible develop ; la version stable est promue vers main. La migration des versions precedentes dans main reste visible dans l'historique Git. La CI est declenchee par les PR et les push sur main/develop.

Le travail DevOps est prepare dans la PR #8. Le depot conserve les contributions des cinq membres, avec des commits distincts et lisibles.

3. Pipeline : etapes et criteres de passage

Etape	Execution	Preuve / critere
1. Installer	Checkout du commit ; Composer et npm a partir des fichiers lock.	Versions PHP/Node definies ; controles Composer et extensions.
2. Preparer	Creation et migration de bases MySQL jetables ; donnees de demonstration en CI.	Aucune connexion a la base de production ; donnees de test isolees.
3. Construire	Compilation Vite et controle du format PHP avec Pint.	Le build et le formatage doivent reussir.
4. Tester	Meme suite sur SQLite et MySQL ; generation Clover/JUnit.	Tous les tests passent ; rapports conserves 14 jours.
5. Analyser	SonarQube Cloud importe le code et la couverture.	Le scanner attend le Quality Gate ; son echec bloque le job.
6. Packager	Docker multi-stage ; migration et demarrage d'un conteneur en CI.	/up, /, /login et /catalog : HTTP 200. Chemins prives : HTTP 404.
7. Deployer	Railway attend la CI sur main ; reconstruit le Dockerfile ; migrate --force.	La migration doit reussir ; /up doit etre sain avant le trafic.
8. Exploiter	Controle HTTPS, consultation des logs et des ressources.	Controle manuel apres livraison et avant la validation.

Preuves accessibles

Execution GitHub Actions
Projet SonarQube Cloud
Code du pipeline et historique

CI : In progress: new coverage, SonarQube quality gate and Docker job.
SonarQube : CI analysis is being verified; automatic analysis was replaced by GitHub Actions.

Les Actions tierces sont epinglees par SHA. Le token Sonar est chiffre dans GitHub Actions ; il ne figure pas dans le code ni dans ce dossier. La couverture est importee depuis les tests, elle n'est pas inventee.

4. Architecture et deploiement

Composant	Responsabilite
Service application	Image Docker PHP 8.4 / Apache ; seul public/ est expose. Les assets Vite et les dependances PHP de production sont inclus.
Service MySQL	Base hebergee distincte des bases locales et de CI. Volume persistant pour les donnees.
Volume application	/var/www/html/storage/app : logos, photos et preuves prives/publics. Seul app/public est relie au web.
Variables privees	APP_KEY stable, connexion MySQL, cles IA et eventuel SMTP. Configuration de production : debug desactive, cookies HTTPS, proxy configure.
Avant demarrage	Railway execute migrate --force. Aucun migrate:fresh et aucun seeder automatique en CD.
Demarrage	Creation des dossiers, permissions, lien public, caches Laravel, puis Apache.
Etat de sante	/up verifie Laravel et une requete SQL. Une panne de base retourne HTTP 500 sans exposer les details de connexion.

Etat du deploiement

Pending: Limited Trial account, 0 hosted services created so far.

Aucune URL de deploiement validee au moment de cette edition. La configuration et la preuve locale Docker sont disponibles.

Supervision, sauvegarde et retour arriere

La plateforme expose les logs de deploiement, CPU/RAM/reseau et les versions precedentes. /up est controle avant la mise en ligne ; ce controle n'est pas un service de supervision continue. Avant une modification destructive, exporter une sauvegarde MySQL et conserver les fichiers uploades.

En cas d'incident : lire les logs, arreter la promotion, redemarrer ou redeployer la derniere version validee. Revenir a une image precedente n'annule pas une migration SQL : conserver des migrations compatibles et une sauvegarde. Aucun Grafana/Prometheus n'est annonce comme installe.

5. Scenario de validation et limites

Demonstration	Ce qu'il faut montrer
Collaboration	Historique des contributions des 5 membres, branches, PR et version main.
CI	Une execution verte, les deux bases de test, le build frontend et les rapports telechargeables.
Qualite	Resultat SonarQube, couverture importee et comportement du Quality Gate. Ne pas confondre une analyse sans condition et un resultat mesure.
Docker	Dockerfile, services application/MySQL, ports, volumes et page /up.
CD	Release Railway, commit deploye, migrations et controle de sante ; URL publique si disponible.
Exploitation	Logs, ressources et procedure de retour a une version precedente.
Application	Demonstration locale des roles, CRUD, controles serveur, fonctions avancees et IA de chaque module.

Limites declarees

La demonstration hebergee peut contenir moins de donnees et d'integrations que la version locale. Le Limited Trial Railway peut limiter les appels IA/SMTP ; l'application locale complete sert de demonstration fonctionnelle. Les credits et la disponibilite de la plateforme doivent etre verifies avant la seance. Aucun plan payant n'est active sans decision du proprietaire.

Les tests automatises valident les droits, les formulaires, les relations et la gestion des erreurs ; ils ne garantissent pas la disponibilite des services externes. Les trois assistants Groq ont aussi fait l'objet d'un test reel ; aucun e-mail n'a ete envoye pendant le controle SMTP.

References techniques

Laravel 12 - deploiement
Railway - attendre la CI
Railway - commande avant deploiement
Railway - limites du trial
SonarQube Cloud - GitHub Actions

Le fichier handover.pdf contient les commandes de reprise de l'equipe et les actions restantes. Ce dossier peut etre envoye a l'enseignant avant la validation ; il ne contient ni mot de passe prive ni cle API.